

MAXIME BRONNY

19009314

TECHNIQUES D'APPRENTISSAGE ARTIFICIEL

- RAPPORT EXPLICATIF -

À || PRÉDICTION DU RISQUE DE CRÉDIT BANCAIRE PAR RÉGRESSION LOGISTIQUE,
ARBRE DE DÉCISION CART ET K-NN IMPLÉMENTATIONS FROM SCRATCH EN PYTHON

PRÉSENTATION

Système de prédiction du risque de défaut de crédit développé en Python avec numpy, reposant sur trois méthodes d'apprentissage supervisé implémentées from scratch : une Régression Logistique, un Arbre de Décision CART et un algorithme des k plus proches voisins, le k-NN. scikit-learn est utilisé uniquement pour vérifier les implémentations.

Les données proviennent du Credit Risk Dataset de Kaggle. Le jeu contient 32 581 emprunteurs et 12 variables, dont 8 numériques et 4 catégorielles. Les classes sont déséquilibrées, avec environ 22 % de défauts.

Le pipeline couvre le chargement, la validation, le nettoyage, l'encodage des variables catégorielles et le prétraitement sans fuite de données. Il inclut une imputation médiane, une normalisation ajustée uniquement sur l'ensemble d'entraînement, puis un découpage stratifié en 70 % entraînement, 15 % validation et 15 % test.

Les trois modèles sont entraînés avec une sélection des hyperparamètres sur l'ensemble de validation, selon le critère F1, puis évalués sur l'ensemble de test. L'Arbre de Décision CART, avec une profondeur de 7, obtient les meilleurs résultats : 93,14 % d'accuracy, 97,65 % de précision, 90,86 % d'AUC ROC et 81,72 % de F1-score. Il devance le k-NN, avec $k = 31$, ainsi que la Régression Logistique.

Les implémentations sont validées par comparaison avec scikit-learn sur l'AUC ROC. L'écart reste inférieur ou égal à 0,0005 pour les trois modèles, avec une AUC identique pour l'Arbre de Décision et le k-NN. Le projet montre ainsi l'intérêt d'une implémentation from scratch pour comprendre les algorithmes, comparer des approches complémentaires et appliquer une évaluation rigoureuse.

[HTTPS://GITHUB.COM/CRYZNIIX/CREDIT-
RISK-PREDICTION/TREE/MAIN](https://github.com/CRYZNIIX/CREDIT-RISK-PREDICTION/TREE/MAIN)

UNIVERSITÉ
PARIS8
VINCENNES-SAINT-DENIS

Sommaire

Liste des sigles et glossaire	iii
1 Introduction	1
2 Contexte et problématique	2
2.1 Le problème métier	2
2.2 Formulation en problème d'apprentissage	2
2.3 Problématique	2
3 Présentation des données	3
3.1 Source et vue d'ensemble	3
3.2 Description des variables	3
3.3 Variable cible et déséquilibre	3
3.4 Problèmes identifiés lors de l'exploration	4
4 Prétraitement des données	5
4.1 Nettoyage des valeurs aberrantes	5
4.2 Encodage des variables catégorielles	5
4.3 Découpage entraînement / validation / test	6
4.4 Imputation des valeurs manquantes	7
4.5 Normalisation	7
5 Méthodologie	8
5.1 Choix des algorithmes	8
5.2 Protocole expérimental	8
5.3 Métriques d'évaluation	10
6 Algorithmes implémentés	11
6.1 Régression logistique	11
6.2 Arbre de décision CART	12
6.3 k plus proches voisins (k -NN)	13
7 Résultats	14

7.1	Sélection des hyperparamètres sur la validation	14
7.2	Performances sur l'ensemble de test	14
7.3	Matrices de confusion	15
7.4	Courbes ROC et comparaison globale	16
7.5	Validation des implémentations avec scikit-learn	17
8	Analyse critique	18
8.1	Pourquoi l'arbre de décision domine-t-il ?	18
8.2	L'apport mesurable de l'ensemble de validation	18
8.3	Surapprentissage : un risque contrôlé	18
8.4	Le rappel, limite commune aux trois modèles	19
8.5	Points forts et réserves sur la qualité des données	19
9	Limites et améliorations possibles	20
10	Conclusion	22
10.1	Bilan du travail réalisé	22
10.2	Réponse à la problématique	22
10.3	Intérêt de la migration vers Python	22
10.4	Bilan personnel d'apprentissage	23
	Bibliographie	24

Liste des sigles et glossaire

Accuracy	Exactitude : proportion de prédictions correctes, toutes classes confondues, $(TP + TN)/N$. Peu informative seule sur des données déséquilibrées.
Apprentissage supervisé	Famille de méthodes qui apprennent à prédire une variable cible à partir d'exemples étiquetés.
Arbre de décision	Modèle qui partitionne récursivement les données par des tests binaires « variable $\leq$ seuil » et prédit, dans chaque feuille, la proportion de défauts observée (voir CART).
AUC	<i>Area Under the Curve</i> : aire sous la courbe ROC ; probabilité qu'un défaut tiré au hasard reçoive un score plus élevé qu'un non-défaut (0,5 = hasard, 1 = classement parfait).
CART	<i>Classification And Regression Tree</i> : algorithme d'arbre de décision fondé sur des séparations binaires successives des données.
Classification binaire	Tâche supervisée où la cible ne prend que deux valeurs ; ici, 0 = remboursement, 1 = défaut de paiement.
Courbe ROC	Courbe du taux de vrais positifs en fonction du taux de faux positifs lorsque le seuil de décision varie (voir ROC, AUC).
Dataset	Jeu de données : ensemble des observations disponibles ; ici, 32 581 dossiers de crédit décrits par 11 variables et une cible.
Descente de gradient	Algorithme d'optimisation itératif qui ajuste les paramètres dans la direction qui fait le plus baisser la fonction de coût.
Encodage	Conversion des variables catégorielles en nombres ; nous utilisons un <i>label encoding</i> à correspondance fixe.
F1-score	Moyenne harmonique de la précision et du rappel ; critère de sélection des hyperparamètres dans ce projet.
Faux négatif (FN)	Défaut réel non détecté (crédit accordé à un client qui ne remboursera pas) : l'erreur la plus coûteuse pour la banque.
Faux positif (FP)	Bon client classé à tort comme défaut (crédit refusé sans raison réelle).
FN	<i>Faux négatif</i> : voir Faux négatif.
FP	<i>Faux positif</i> : voir Faux positif.
From scratch	Implémenté à partir des opérations de base (NumPy), sans bibliothèque d'apprentissage automatique.

Fuite de données	<i>Data leakage</i> : utilisation, même indirecte, d'informations de validation ou de test pendant la conception du modèle ; elle conduit à surestimer les performances.
Généralisation	Capacité d'un modèle à rester performant sur des données jamais vues ; c'est ce que mesure l'ensemble de test.
Hyperparamètre	Réglage fixé avant l'apprentissage (profondeur de l'arbre, nombre de voisins k , seuil de décision), choisi ici sur l'ensemble de validation.
Imputation	Remplacement des valeurs manquantes par une valeur calculée ; ici, la médiane de l'ensemble d'entraînement.
Impureté de Gini	Critère de qualité d'une division de l'arbre : $G = 2p(1 - p)$ pour un groupe contenant une proportion p de défauts.
Jeu d'entraînement	Sous-ensemble (70 %) servant à apprendre les paramètres des modèles.
Jeu de test	Sous-ensemble (15 %) utilisé une seule fois, à la fin, pour mesurer la performance de généralisation.
Jeu de validation	Sous-ensemble (15 %) servant à choisir les hyperparamètres.
k-NN	<i>k plus proches voisins (k-Nearest Neighbors)</i> : méthode qui classe un exemple par vote des k exemples d'entraînement les plus proches au sens de la distance euclidienne.
Log-loss	Entropie croisée : fonction de coût de la régression logistique, mesurant l'écart entre probabilités prédites et étiquettes réelles.
Matrice de confusion	Tableau croisant classes réelles et classes prédites (TP, TN, FP, FN).
Modèle	Fonction apprise des données qui associe une prédiction à un vecteur de variables explicatives.
Normalisation	Mise à l'échelle de chaque variable (score z) : soustraction de la moyenne puis division par l'écart-type, calculés sur l'ensemble d'entraînement.
NumPy	Bibliothèque Python de calcul numérique (tableaux, algèbre linéaire) ; base de toutes les implémentations <i>from scratch</i> du projet.
Précision	Parmi les défauts prédits, proportion de défauts réels : $TP/(TP + FP)$.
Prétraitement	Transformations appliquées aux données brutes avant l'apprentissage : nettoyage, encodage, imputation, normalisation.
Python	Langage de programmation utilisé pour l'ensemble du projet.
Rappel	Parmi les défauts réels, proportion détectée : $TP/(TP + FN)$.
Régression logistique	Modèle linéaire qui estime la probabilité de défaut en passant le score $\mathbf{w} \cdot \mathbf{x} + b$ dans la sigmoïde.
ROC	<i>Receiver Operating Characteristic</i> : voir Courbe ROC.

Scikit-learn	Bibliothèque Python d'apprentissage automatique, utilisée ici uniquement pour vérifier la justesse des implémentations <i>from scratch</i> .
Seuil de décision	Valeur au-delà de laquelle la probabilité prédite est convertie en classe « défaut ».
Sigmoïde	Fonction $\sigma(z) = 1/(1 + e^{-z})$, qui transforme un score réel en probabilité comprise entre 0 et 1.
Stratification	Découpage effectué classe par classe afin que chaque ensemble conserve la même proportion de défauts (21,8 %).
Surapprentissage	<i>Overfitting</i> : situation où le modèle colle au bruit de l'entraînement et généralise mal à de nouvelles données.
TN	<i>True Negative</i> (vrai négatif) : remboursement correctement identifié.
TP	<i>True Positive</i> (vrai positif) : défaut correctement détecté.
Variable cible	Valeur à prédire (<code>loan_status</code>) : 1 = défaut, 0 = remboursement.
Variable explicative	Entrée du modèle (âge, revenu, montant, taux, etc.) à partir de laquelle il prédit la cible.

1. Introduction

L'évaluation du risque de crédit est l'une des activités les plus sensibles du secteur bancaire : chaque prêt accordé repose sur un pari, celui que l'emprunteur remboursera. Historiquement confiée à des analystes humains s'appuyant sur des grilles de notation, cette évaluation est aujourd'hui largement automatisée grâce à l'apprentissage automatique, qui permet d'exploiter les données historiques de remboursement pour estimer la probabilité de défaut d'un nouveau client. Le *credit scoring* constitue ainsi l'un des cas d'usage les plus anciens et les mieux documentés de la classification supervisée [5].

Ce projet, réalisé dans le cadre du cours de Techniques d'Apprentissage Artificiel, a pour objectif de construire un pipeline complet d'apprentissage automatique appliqué à la prédiction du risque de crédit, puis de comparer rigoureusement trois méthodes de classification de familles différentes : la régression logistique, l'arbre de décision CART et les k plus proches voisins (k -NN). Un parti pris fort structure l'ensemble du travail : les trois algorithmes, ainsi que toutes les métriques d'évaluation, sont implémentés *from scratch* en Python [10], à l'aide de la seule bibliothèque de calcul numérique NumPy [4]. La bibliothèque scikit-learn [9] n'intervient qu'en fin de chaîne, uniquement comme référence permettant de *vérifier* la justesse de nos implémentations. Elle n'est jamais utilisée comme solution.

Précisons un point d'historique : la première version de ce projet avait été développée en langage C, avec un découpage train/test simple et deux modèles. Le projet a été entièrement réécrit en Python pour répondre aux consignes de la matière ; cette réécriture a été l'occasion d'enrichir le protocole expérimental (ajout d'un ensemble de validation dédié au choix des hyperparamètres, ajout d'une troisième méthode) et de valider systématiquement les implémentations par comparaison avec scikit-learn. L'ancienne version est conservée à titre d'archive mais n'intervient plus dans le présent travail.

Ce rapport est organisé comme suit. La section 2 présente le contexte métier et la problématique. La section 3 décrit le jeu de données utilisé et la section 4 les étapes de prétraitement, dont le découpage entraînement/validation/test. La section 5 détaille le protocole expérimental et les métriques retenues, puis la section 6 expose le principe et l'implémentation de chacun des trois algorithmes. Les résultats sont présentés en section 7, analysés de manière critique en section 8, et la section 9 discute les limites du travail et les améliorations envisageables, avant la conclusion.

2. Contexte et problématique

2.1. Le problème métier

Lorsqu'une banque accorde un crédit, deux types d'erreurs sont possibles, et leurs coûts sont très asymétriques. Si la banque prête à un client qui ne remboursera pas (défaut non détecté), elle perd tout ou partie du capital prêté ; si elle refuse un client qui aurait remboursé, elle ne perd qu'un manque à gagner, les intérêts du prêt. Un bon système de prédiction du risque doit donc non seulement être globalement précis, mais surtout détecter le plus possible de futurs défauts, sans pour autant rejeter massivement les bons dossiers.

Le volume des demandes de crédit rend l'examen entièrement manuel impraticable, tandis que les banques disposent d'historiques de remboursement riches et structurés. Ce contexte, caractérisé par beaucoup de données étiquetées et une décision binaire à automatiser, constitue un cadre idéal pour l'apprentissage supervisé.

2.2. Formulation en problème d'apprentissage

Le problème se formule comme une **classification binaire supervisée** : à partir d'un vecteur de caractéristiques décrivant l'emprunteur et son prêt (âge, revenu, montant, taux d'intérêt, historique de crédit, etc.), prédire la variable cible `loan_status` :

- classe 0 : remboursement normal (environ 78 % des observations) ;
- classe 1 : défaut de paiement (environ 22 % des observations).

Ce déséquilibre des classes est la difficulté centrale du problème : un modèle naïf qui prédirait systématiquement « pas de défaut » atteindrait déjà environ 78 % d'exactitude tout en étant parfaitement inutile, puisqu'il ne détecterait aucun défaut. L'évaluation devra donc reposer sur des métriques adaptées (précision, rappel, F1-score, courbe ROC), présentées en section 5.

2.3. Problématique

La problématique retenue pour ce projet est la suivante :

À partir des caractéristiques d'un emprunteur et de son prêt, peut-on prédire automatiquement le risque de défaut de paiement, et quelle méthode d'apprentissage supervisé (une méthode linéaire, une méthode à base de règles ou une méthode à base d'instances) offre le meilleur compromis entre la détection des défauts et les erreurs de classification ?

Répondre à cette question suppose un protocole expérimental rigoureux : mêmes données, même prétraitement et même méthode de sélection des hyperparamètres pour les trois modèles, et une évaluation finale sur des données jamais utilisées pendant la conception.

3. Présentation des données

3.1. Source et vue d'ensemble

Nous utilisons le *Credit Risk Dataset* publié sur la plateforme Kaggle [7] (fichier `data/raw/credit_risk_dataset`). Il contient **32 581 observations**, chacune décrivant un emprunteur et le prêt qui lui a été accordé, au travers de **11 variables explicatives** et d'une variable cible binaire.

3.2. Description des variables

Le tableau 2 récapitule les variables du jeu de données.

Table 2 - Variables du *Credit Risk Dataset*.

Variable	Type	Description
person_age	numérique	âge de l'emprunteur (années)
person_income	numérique	revenu annuel
person_home_ownership	catégorielle (4)	logement : location, propriétaire, hypothèque, autre
person_emp_length	numérique	ancienneté d'emploi (années)
loan_intent	catégorielle (6)	objet du prêt : personnel, études, médical, entreprise, travaux, rachat de dettes
loan_grade	catégorielle ordinale (A-G)	note de risque attribuée au dossier (A = meilleur)
loan_amnt	numérique	montant du prêt
loan_int_rate	numérique	taux d'intérêt (%)
loan_percent_income	numérique	part du revenu consacrée au prêt
cb_person_default_on_file	catégorielle (O/N)	antécédent de défaut dans l'historique de crédit
cb_person_cred_hist_length	numérique	longueur de l'historique de crédit (années)
loan_status	cible (0/1)	1 = défaut de paiement, 0 = remboursement

3.3. Variable cible et déséquilibre

La cible `loan_status` vaut 1 pour 21,8 % des observations : le jeu de données est déséquilibré, dans des proportions toutefois courantes pour ce type de problème (figure 1). Ce déséquilibre guide le choix des métriques (section 5) et justifie le découpage stratifié des données (section 4).

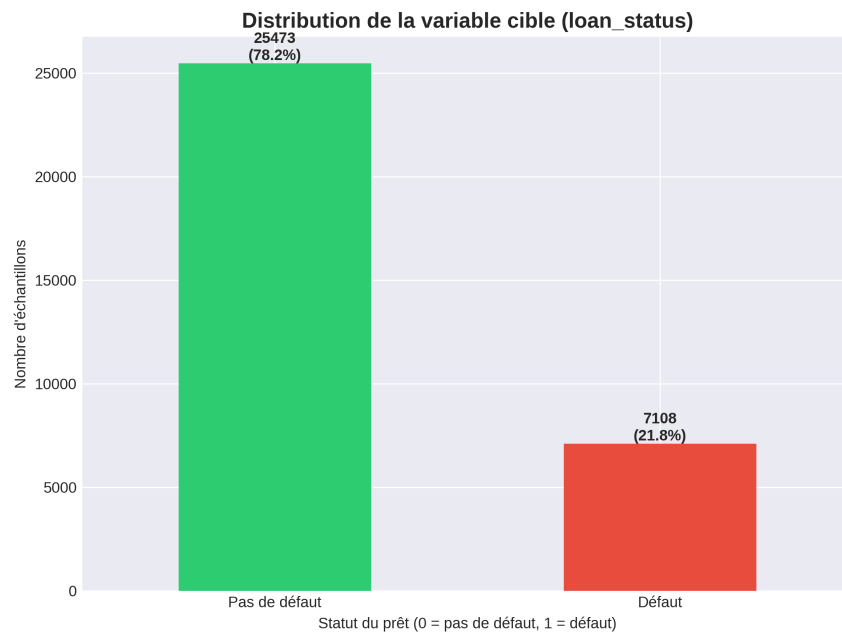


Figure 1 - Distribution de la variable cible `loan_status` : le jeu de données est déséquilibré (21,8 % de défauts).

3.4. Problèmes identifiés lors de l'exploration

L'analyse exploratoire (figures et statistiques disponibles dans `data/stats/`) a mis en évidence trois problèmes à traiter avant l'apprentissage :

- **valeurs manquantes** : deux variables en contiennent, `loan_int_rate` (9,6 % des lignes) et `person_emp_length` (2,75 %) ;
- **valeurs aberrantes** : quelques lignes contiennent des valeurs physiquement impossibles, comme des âges supérieurs à 120 ans ou une ancienneté d'emploi de 123 ans, manifestement des erreurs de saisie ;
- **déséquilibre des classes** : 78/22, discuté ci-dessus.

L'exploration montre par ailleurs des corrélations attendues : le taux de défaut croît fortement avec le grade du prêt (de A vers G) et avec la part du revenu consacrée au remboursement (`loan_percent_income`), deux variables dont on peut anticiper qu'elles joueront un rôle important dans les modèles. Aucune colonne n'a été jugée inutile : les 11 variables explicatives sont conservées.

4. Prétraitement des données

Le prétraitement, qui s'appuie sur la bibliothèque pandas [8] pour le chargement et la manipulation des données, est implémenté dans `src/preprocessing.py` et `src/split_data.py`. Un principe structure l'ensemble : **éviter toute fuite de données** (*data leakage*). Tout paramètre appris des données (médianes d'imputation, moyennes et écarts-types de normalisation) est calculé *uniquement* sur l'ensemble d'entraînement, puis appliqué tel quel aux ensembles de validation et de test, qui jouent ainsi pleinement leur rôle de données « jamais vues ».

4.1. Nettoyage des valeurs aberrantes

Les lignes physiquement impossibles identifiées lors de l'exploration sont retirées : âge supérieur à 100 ans ou ancienneté d'emploi supérieure à 60 ans. Seules **7 lignes sur 32 581** sont concernées (32 574 lignes conservées). Nous ne touchons volontairement pas aux valeurs extrêmes mais plausibles (revenus très élevés par exemple) : elles font partie de la vraie distribution des données que les modèles doivent apprendre à gérer.

4.2. Encodage des variables catégorielles

Les quatre variables catégorielles sont converties en entiers par un *label encoding* à correspondance fixe, définie dans le code et non apprise des données (tableau 3) ; l'encodage est donc identique quel que soit le découpage.

Table 3 - Encodage des variables catégorielles.

Variable	Correspondance
<code>loan_grade</code>	A→0, B→1, ..., G→6
<code>cb_person_default_on_file</code>	N→0, Y→1
<code>person_home_ownership</code>	RENT→0, OWN→1, MORTGAGE→2, OTHER→3
<code>loan_intent</code>	PERSONAL→0, EDUCATION→1, MEDICAL→2, VENTURE→3, HOMEIMPROVEMENT→4, DEBTCONSOLIDATION→5

Ce choix mérite justification. Pour `loan_grade`, l'encodage entier est naturel : la variable est *ordinaire* (A est un meilleur dossier que G) et l'ordre numérique reflète un ordre réel. Pour `cb_person_default_on_file`, binaire, l'encodage est sans ambiguïté. Pour les deux variables *nominales* (`person_home_ownership` et `loan_intent`), le label encoding introduit en revanche un ordre artificiel entre modalités. Ce choix, hérité de la première version du projet et conservé pour permettre la comparaison, reste acceptable pour l'arbre de décision (qui peut isoler chaque modalité par divisions successives) et pour le *k*-NN, mais constitue une limite pour la régression logistique ; nous y revenons dans l'analyse critique (section 8).

4.3. Découpage entraînement / validation / test

Le jeu de données nettoyé est découpé en trois ensembles disjoints : **70 % pour l'entraînement**, **15 % pour la validation** et **15 % pour le test**, soit respectivement **22 802**, **4 886** et **4 886** observations. Chaque ensemble a un rôle strictement distinct :

- l'**entraînement** sert à apprendre les paramètres des modèles (poids de la régression, structure de l'arbre, instances mémorisées du k -NN) ;
- la **validation** sert *uniquement* à choisir les hyperparamètres (seuil de décision, profondeur de l'arbre, nombre de voisins k) ;
- le **test** n'est utilisé qu'*une seule fois*, à la toute fin, pour mesurer la performance de généralisation.

Cette séparation en trois est essentielle : si les hyperparamètres étaient choisis directement sur le test, celui-ci ne mesurerait plus une capacité de généralisation mais un ajustement indirect aux données d'évaluation, et les performances rapportées seraient optimistes. La répartition 70/15/15 offre un bon compromis pour un jeu de cette taille : assez d'exemples d'entraînement pour des modèles stables, et près de 5 000 exemples dans chacun des deux ensembles d'évaluation, ce qui donne des estimations fiables.

Le découpage est **stratifié** : il est effectué classe par classe, de sorte que le taux de défaut (21,8 %) soit identique dans les trois ensembles, comme le confirme la figure 2. Sans stratification, un tirage aléatoire pourrait produire un ensemble de test non représentatif. Le mélange utilise une graine aléatoire fixe (42), garantissant la reproductibilité de toute l'expérience.

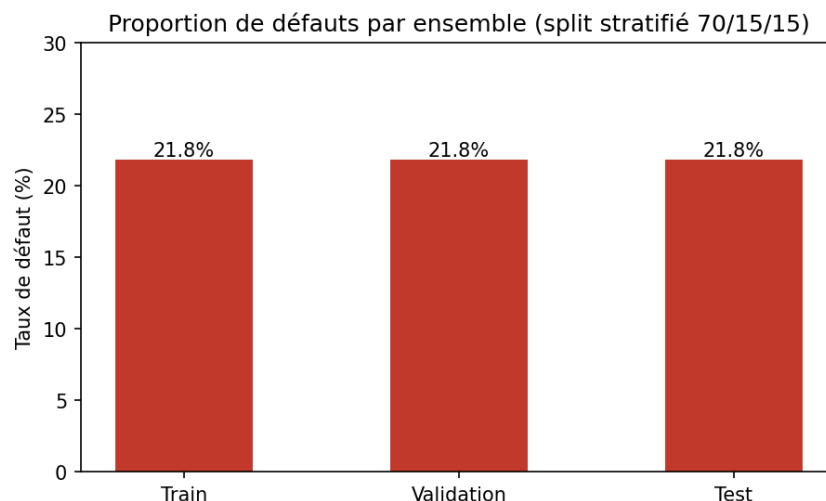


Figure 2 - Taux de défaut par ensemble après découpage stratifié 70/15/15 : les trois ensembles conservent la même proportion de défauts (21,8 %).

4.4. Imputation des valeurs manquantes

Les valeurs manquantes des variables numériques sont remplacées par la **médiane** de la variable, calculée sur le seul ensemble d'entraînement. La médiane est préférée à la moyenne car elle est robuste aux valeurs extrêmes, nombreuses dans les variables de revenu et de montant.

4.5. Normalisation

Les variables sont enfin standardisées (score z) :

$$x' = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}},$$

où μ_{train} et σ_{train} sont la moyenne et l'écart-type calculés sur l'entraînement. Cette normalisation est indispensable pour deux des trois modèles : pour la régression logistique, elle conditionne la vitesse et la stabilité de la descente de gradient ; pour le k -NN, elle évite qu'une variable à grande échelle comme le revenu (de l'ordre de 10^4 à 10^5) n'écrase toutes les autres dans le calcul des distances. L'arbre de décision y est insensible (il ne compare que des seuils par variable), mais reçoit les mêmes données afin de conserver un pipeline unique.

5. Méthodologie

5.1. Choix des algorithmes

Nous comparons trois méthodes de classification appartenant à trois familles distinctes, ce qui donne à la comparaison un intérêt qui dépasse le simple classement de performances :

- la **régression logistique**, modèle *linéaire* probabiliste : c'est la méthode de référence historique du credit scoring [5], interprétable (chaque variable reçoit un poids) et rapide ;
- l'**arbre de décision CART** [1], modèle *non linéaire* à base de règles : il capture les interactions entre variables et produit des décisions lisibles (suites de tests « si... alors ») ;
- les **k plus proches voisins** [2], méthode *non paramétrique* à base d'instances : aucune phase d'apprentissage, la prédiction repose entièrement sur la similarité aux exemples connus.

Nous avons volontairement limité l'étude à trois méthodes maîtrisées de bout en bout (toutes trois implémentées from scratch) plutôt que d'en survoler davantage : l'objectif pédagogique est de comprendre finement chaque algorithme, ses hyperparamètres et ses modes d'échec.

5.2. Protocole expérimental

Le protocole, entièrement automatisé par `main.py`, est identique pour les trois modèles :

1. **entraînement** sur les 22 802 exemples du train ;
2. **sélection des hyperparamètres sur la validation** (4 886 exemples), avec le F1-score comme critère :
 - seuil de décision (tous les modèles) : grille {0,3; 0,4; 0,5; 0,6} ;
 - profondeur maximale de l'arbre : grille {3, 5, 7, 9} ;
 - nombre de voisins du k -NN : grille {5, 11, 21, 31} ;
3. **évaluation finale unique** sur les 4 886 exemples du test ;
4. **vérification** : chaque modèle est ré-entraîné avec son équivalent scikit-learn [9] sur les mêmes données et avec des hyperparamètres équivalents, afin de valider la justesse de nos implémentations. Cette comparaison joue un rôle de contrôle qualité, pas de solution.

Les grilles sont volontairement compactes : il s'agit de démontrer la démarche de sélection sur validation, pas de mener une recherche exhaustive. L'expérience complète est reproductible (graine fixe, une seule commande) et s'exécute en une vingtaine de secondes.

La figure 3 résume l'enchaînement complet des scripts, de la lecture des données brutes jusqu'à l'intégration des résultats dans ce rapport, en distinguant l'ordre d'exécution (flèche pleine) du flux de données (flèche pointillée).

Projet TAA - Schéma d'exécution des scripts

Pipeline Python - données, modèles, métriques et rapport

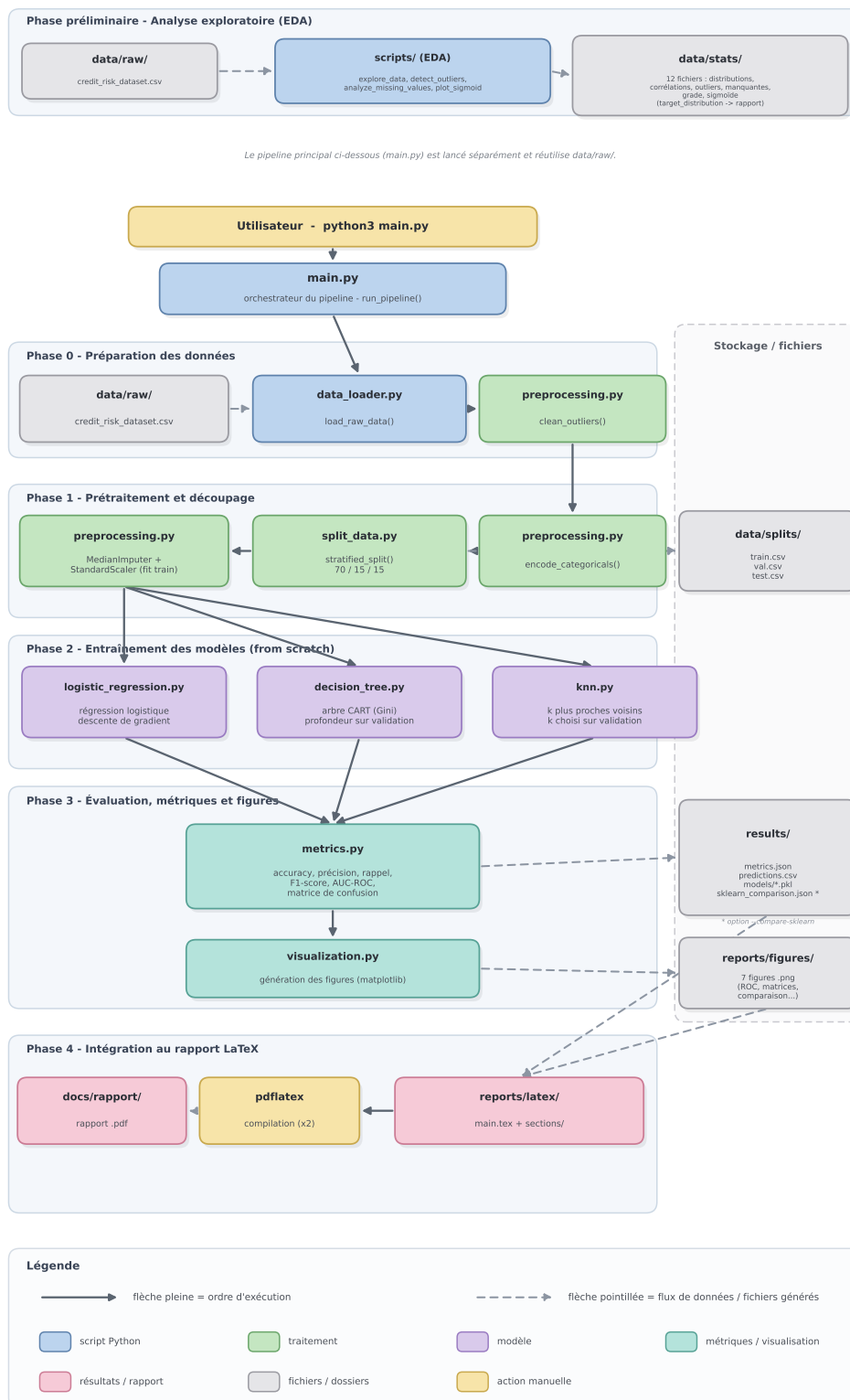


Figure 3 - Schéma d'exécution des scripts du projet : enchaînement du pipeline Python, des données aux modèles, puis aux métriques, figures et rapport.

5.3. Métriques d'évaluation

Toutes les métriques sont implémentées from scratch (`src/metrics.py`). En notant TP, TN, FP, FN les vrais/faux positifs/négatifs (la classe positive étant le défaut) :

$$\begin{aligned}\text{accuracy} &= \frac{TP + TN}{TP + TN + FP + FN} & \text{précision} &= \frac{TP}{TP + FP} \\ \text{rappel} &= \frac{TP}{TP + FN} & F_1 &= \frac{2 \cdot \text{précision} \cdot \text{rappel}}{\text{précision} + \text{rappel}}\end{aligned}$$

Le choix de ces métriques découle directement du déséquilibre des classes :

- l'**accuracy** donne une vue d'ensemble mais est insuffisante seule (prédire systématiquement la classe majoritaire donne déjà 78%) ;
- la **précision** mesure la fiabilité des alertes : parmi les défauts prédits, combien sont réels ;
- le **rappel** mesure la couverture : parmi les vrais défauts, combien sont détectés. C'est la métrique critique pour la banque, puisqu'un faux négatif correspond à un crédit accordé à un client qui ne remboursera pas ;
- le **F1-score**, moyenne harmonique des deux précédentes, sert de critère de sélection des hyperparamètres ;
- la **matrice de confusion** permet d'analyser la nature des erreurs ;
- la **courbe ROC** et l'**aire sous la courbe (AUC)** [3] comparent les modèles indépendamment du seuil de décision, à partir des probabilités prédites : l'AUC s'interprète comme la probabilité qu'un défaut tiré au hasard reçoive un score plus élevé qu'un non-défaut tiré au hasard.

Les métriques de régression (MAE, RMSE, R^2) et de partitionnement (inertie, silhouette) ne sont pas pertinentes ici : le problème est une classification supervisée, ni une régression ni un clustering.

6. Algorithmes implémentés

Les trois algorithmes sont implémentés from scratch en Python, avec NumPy pour le calcul vectoriel, dans le module `src/models/`. Nous présentons pour chacun le principe, les équations, les choix d'implémentation et les limites.

6.1. Régression logistique

Principe. La régression logistique [5] modélise directement la probabilité de défaut comme une fonction du score linéaire $z = \mathbf{w} \cdot \mathbf{x} + b$ passée dans la sigmoïde :

$$p(y = 1 \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Les paramètres $\mathbf{w}$ et b sont appris en minimisant l'entropie croisée (*log-loss*) sur l'ensemble d'entraînement :

$$J(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^n [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)],$$

fonction convexe dont le minimum global est atteignable par descente de gradient. Les gradients ont une forme remarquablement simple :

$$\frac{\partial J}{\partial \mathbf{w}} = \frac{1}{n} X^\top (\mathbf{p} - \mathbf{y}), \quad \frac{\partial J}{\partial b} = \frac{1}{n} \sum_{i=1}^n (p_i - y_i).$$

Implémentation. Notre implémentation (`src/models/logistic_regression.py`) utilise une descente de gradient *full batch* : à chaque itération, le gradient est calculé sur tout l'ensemble d'entraînement en une seule opération matricielle. Deux détails numériques méritent mention : la sigmoïde est calculée par deux formules équivalentes selon le signe de z afin d'éviter tout dépassement de capacité de l'exponentielle, et les probabilités sont bornées avant le logarithme pour éviter $\ln 0$. Les hyperparamètres d'optimisation sont un taux d'apprentissage de 0,1 et 1 000 itérations, sur données normalisées. La figure 4 montre la décroissance régulière de la log-loss : la descente de gradient converge sans oscillation, signe d'un taux d'apprentissage bien dimensionné.

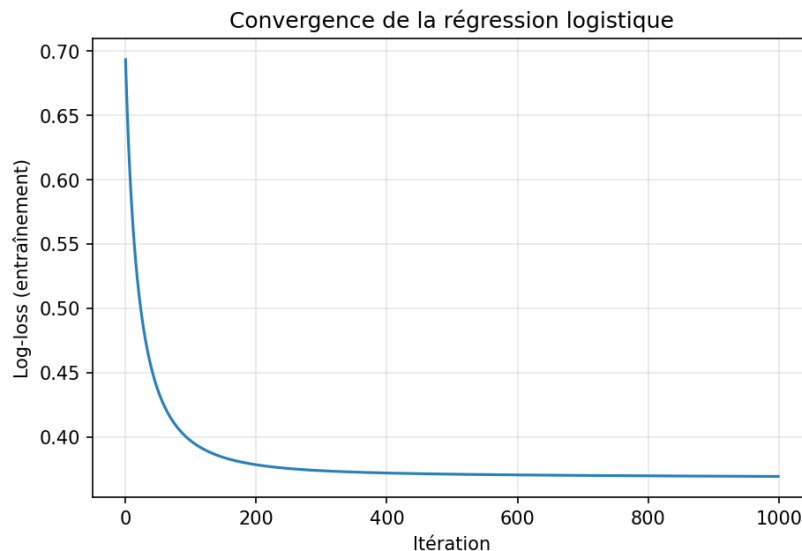


Figure 4 - Convergence de la descente de gradient : décroissance de la log-loss sur l'ensemble d'entraînement (1 000 itérations, taux d'apprentissage 0,1).

Le seuil qui convertit la probabilité en classe est traité comme un hyperparamètre choisi sur la validation ; la valeur retenue, 0,3, est inférieure au seuil naïf de 0,5 : en présence de classes déséquilibrées, abaisser le seuil augmente le rappel, ce qui est cohérent avec le coût élevé des faux négatifs.

Limites. La frontière de décision reste un hyperplan : les interactions entre variables (un même taux d'endettement n'a pas le même sens selon le grade du dossier) ne sont pas modélisées. Le modèle est de plus pénalisé par l'encodage ordinal des variables nominales, qui lui impose un ordre arbitraire.

6.2. Arbre de décision CART

Principe. L'algorithme CART [1] construit un arbre binaire par partitionnement récursif. À chaque nœud, il cherche le couple (variable, seuil) qui sépare le mieux les exemples, au sens de l'impureté de Gini : pour un groupe contenant une proportion p de défauts, $G = 2p(1 - p)$, nulle pour un groupe pur et maximale pour un mélange 50/50. Le couple retenu minimise la moyenne pondérée des impuretés des deux groupes enfants. La construction s'arrête quand le nœud est pur, trop petit, ou que la profondeur maximale est atteinte ; chaque feuille prédit alors la proportion de défauts des exemples qu'elle contient, interprétable comme une probabilité.

Implémentation. La difficulté pratique est le coût de la recherche du meilleur seuil. Notre implémentation (`src/models/decision_tree.py`) la vectorise : pour chaque variable, les valeurs sont triées et les sommes cumulées des étiquettes donnent, pour *toutes* les positions de coupure possibles à la fois, la proportion de défauts de part et d'autre, et donc le Gini de tous les seuils candidats en une seule passe NumPy, sans boucle Python sur les seuils. Cette vectorisation rend l'entraînement quasi instantané sur 22 802 exemples. Deux garde-fous limitent le surapprentissage :

un nœud n'est divisé que s'il contient au moins 20 exemples, et toute feuille doit en contenir au moins 5. La profondeur maximale, hyperparamètre principal, est choisie sur la validation ; l'arbre retenu a une profondeur de 7 et compte 105 nœuds.

Limites. Un arbre seul est un modèle instable : une perturbation des données d'entraînement peut changer les premières divisions et, par effet de cascade, toute la structure. Sans limite de profondeur, il mémoriserait le bruit du train ; le contrôle par la validation est donc indispensable. Enfin, sa frontière de décision « en escalier » approxime grossièrement les relations continues douces.

6.3. k plus proches voisins (k -NN)

Principe. Le k -NN [2] ne construit aucun modèle : l'« apprentissage » consiste à mémoriser l'ensemble d'entraînement. Pour classer un nouvel exemple, on calcule sa distance euclidienne à tous les exemples mémorisés, on retient les k plus proches et l'on prédit la proportion de défauts parmi eux. Le paramètre k règle le compromis biais-variance : petit, la décision est locale mais bruitée ; grand, elle est lissée mais peut noyer les structures locales.

Implémentation. La difficulté est ici la mémoire : la matrice complète des distances entre les 4886 exemples de test et les 22802 exemples d'entraînement occuperait de l'ordre de 900 Mo. Notre implémentation (`src/models/knn.py`) procède donc par paquets de quelques centaines d'exemples. Deux optimisations classiques sont utilisées : le développement $\|a - b\|^2 = \|a\|^2 + \|b\|^2 - 2a \cdot b$, dont le terme $\|a\|^2$, constant pour un exemple donné, peut être omis car il ne change pas l'ordre des voisins ; et la sélection des k plus petites distances par `np.argpartition`, en temps linéaire, au lieu d'un tri complet. La valeur retenue sur la validation est $k = 31$.

Limites. Toute la charge de calcul est reportée à la prédiction, qui parcourt l'ensemble d'entraînement complet pour chaque exemple, ce qui est rédhibitoire pour un système temps réel à fort trafic sans structures d'index dédiées. La méthode est par ailleurs très sensible à l'échelle des variables (d'où la normalisation) et sa pertinence se dégrade en haute dimension.

7. Résultats

7.1. Sélection des hyperparamètres sur la validation

Le tableau 4 retrace la sélection effectuée sur l'ensemble de validation (critère : F1-score). Pour l'arbre, le F1 progresse nettement de la profondeur 3 à la profondeur 7, puis recule légèrement à 9 : au-delà de 7, la complexité supplémentaire n'apporte plus de pouvoir prédictif généralisable. Pour le k -NN, le F1 est remarquablement stable sur la grille, signe que la décision dépend peu du voisinage exact au-delà d'une dizaine de voisins.

Table 4 - Sélection des hyperparamètres sur l'ensemble de validation (F1-score ; en gras, la configuration retenue).

Modèle	Configuration	Seuil optimal	F1 (validation)
Régression logistique	lr = 0,1, 1 000 it.	0,3	0,625
Arbre de décision	profondeur 3	0,3	0,728
	profondeur 5	0,3	0,776
	profondeur 7	0,6	0,819
	profondeur 9	0,6	0,817
k -NN	$k = 5$	0,5	0,683
	$k = 11$	0,4	0,695
	$k = 21$	0,3	0,692
	$k = 31$	0,3	0,695

7.2. Performances sur l'ensemble de test

Le tableau 5 présente l'évaluation finale, réalisée une seule fois sur l'ensemble de test.

Table 5 - Métriques des trois modèles sur l'ensemble de test (4 886 exemples). Hyperparamètres choisis sur la validation.

Modèle	Accuracy	Précision	Rappel	F1-score	AUC-ROC
Régression logistique (seuil 0,3)	0,820	0,573	0,690	0,626	0,848
Arbre de décision (prof. 7, seuil 0,6)	0,931	0,977	0,703	0,817	0,909
k -NN ($k = 31$, seuil 0,3)	0,862	0,683	0,684	0,683	0,877

La hiérarchie est nette et cohérente sur toutes les métriques : **l'arbre de décision domine**, suivi du k -NN, puis de la régression logistique. L'arbre atteint un F1 de 0,817 et une AUC de 0,909 ; son avance la plus spectaculaire concerne la précision (0,977) : lorsqu'il prédit un défaut, il se trompe très rarement. Les trois modèles plafonnent en revanche à un rappel compris entre 0,69 et 0,70 : quel que soit le modèle, environ 30 % des défauts échappent à la détection. Nous y

revenons dans l'analyse critique.

7.3. Matrices de confusion

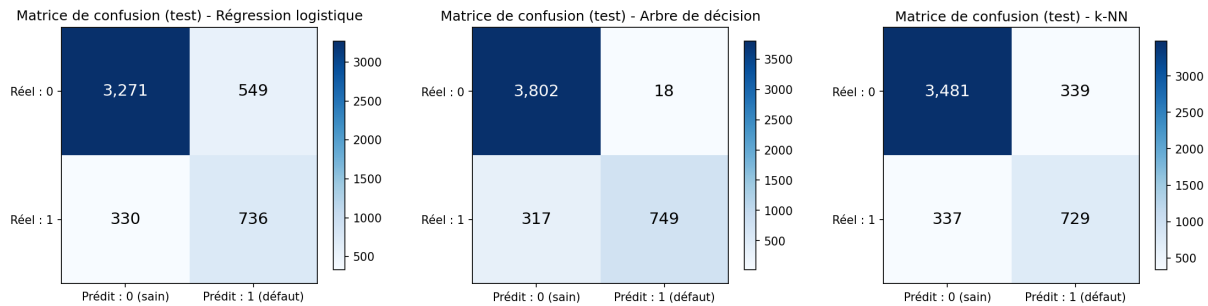


Figure 5 - Matrices de confusion sur l'ensemble de test : régression logistique (gauche), arbre de décision (centre), k -NN (droite).

La lecture détaillée des matrices (figure 5) éclaire les différences de comportement. Sur les 4 886 dossiers de test (dont 1 066 défauts) :

- l'**arbre** ne commet que **18 faux positifs** (bons clients refusés) pour 749 défauts détectés, soit un profil très « prudent dans l'alerte » ; ses erreurs sont presque exclusivement des faux négatifs (317) ;
- la **régression logistique**, avec son seuil abaissé à 0,3, détecte un nombre comparable de défauts (736) mais au prix de 549 faux positifs, soit trente fois plus que l'arbre ;
- le **k -NN** occupe une position intermédiaire (729 défauts détectés, 339 faux positifs).

Autrement dit, à rappel quasi égal, les trois modèles se distinguent presque uniquement par leur taux de fausses alertes. C'est ce qui explique l'écart de précision et donc de F1.

7.4. Courbes ROC et comparaison globale

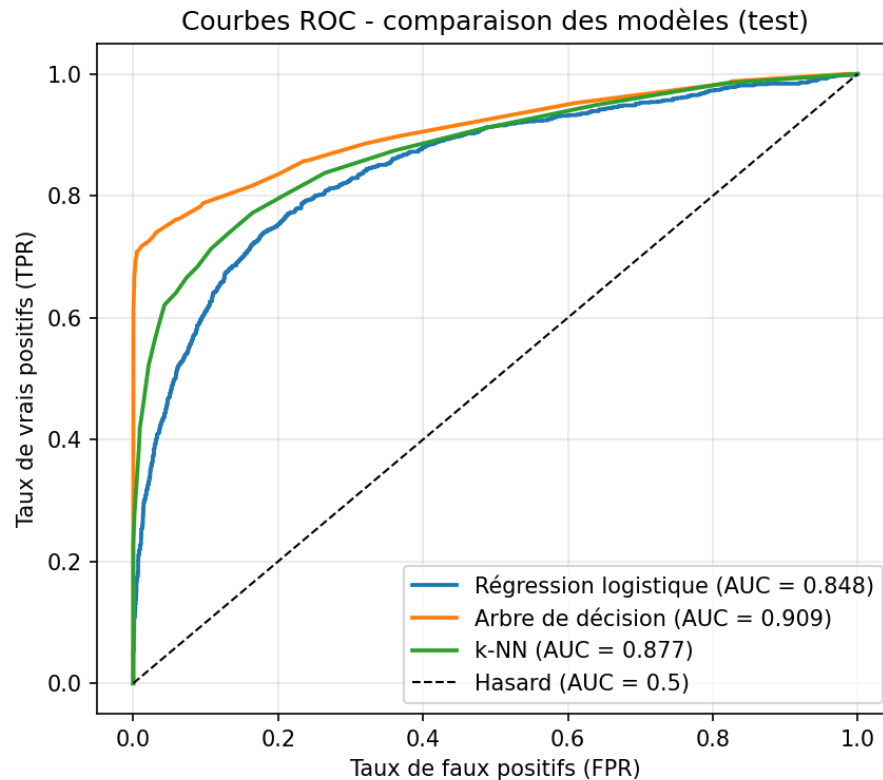


Figure 6 - Courbes ROC des trois modèles sur l'ensemble de test. AUC : arbre de décision 0,909 ; k -NN 0,877 ; régression logistique 0,848.

Les courbes ROC (figure 6) confirment la hiérarchie indépendamment du seuil de décision : la courbe de l'arbre domine celle du k -NN, qui domine celle de la régression logistique, sur la quasi-totalité de la plage de fonctionnement. Les trois modèles sont très au-dessus de la diagonale du hasard (AUC 0,5). La figure 7 récapitule les cinq métriques. L'ensemble des figures de ce rapport est produit avec Matplotlib [6].

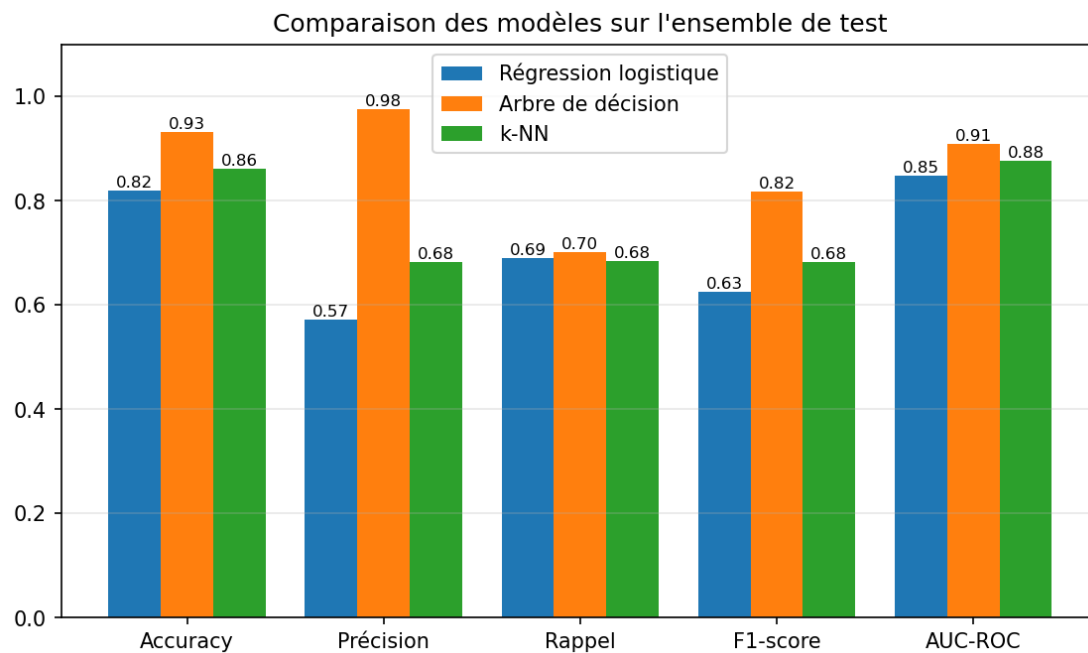


Figure 7 - Comparaison des cinq métriques sur l'ensemble de test.

7.5. Validation des implémentations avec scikit-learn

Pour vérifier la justesse de nos implémentations, chaque modèle a été ré-entraîné avec son équivalent scikit-learn sur exactement les mêmes ensembles et avec des hyperparamètres équivalents (tableau 6).

Table 6 - Vérification des implémentations from scratch : AUC sur l'ensemble de test, comparée à scikit-learn.

Modèle	AUC from scratch	AUC scikit-learn	Écart
Régression logistique	0,8479	0,8483	−0,0005
Arbre de décision	0,9086	0,9086	0,0000
<i>k</i> -NN	0,8772	0,8772	0,0000

L'arbre et le *k*-NN obtiennent une AUC *identique* à celle de scikit-learn, et l'écart de la régression logistique (5×10^{-4}) s'explique par les différences d'optimiseur (notre descente de gradient pure contre L-BFGS) et par la régularisation L2 activée par défaut dans scikit-learn. Ces écarts négligeables valident nos trois implémentations. L'ensemble du pipeline, comprenant le prétraitement, l'entraînement des trois modèles, la sélection sur validation, l'évaluation et la génération des figures, s'exécute en une vingtaine de secondes.

8. Analyse critique

8.1. Pourquoi l'arbre de décision domine-t-il ?

La supériorité de l'arbre s'explique par la nature du problème. Le risque de crédit repose sur des *interactions* entre variables : un taux d'endettement de 30 % n'a pas la même signification pour un dossier de grade A que pour un grade F, et un antécédent de défaut ne pèse pas pareil selon le revenu. La régression logistique, qui additionne des contributions indépendantes des variables, ne peut pas représenter ces effets croisés ; sa frontière de décision reste un hyperplan, et son AUC plafonne à 0,848. L'arbre, en enchaînant des tests conditionnels (d'abord le grade, puis le taux d'endettement *dans* chaque branche, etc.), modélise naturellement ces interactions, d'où son AUC de 0,909 et surtout sa précision de 0,977. Le k -NN, qui capte la structure locale des données sans la modéliser explicitement, se place logiquement entre les deux.

8.2. L'apport mesurable de l'ensemble de validation

Le réglage du seuil de décision sur la validation illustre concrètement l'intérêt du découpage en trois ensembles. Pour la régression logistique, le seuil retenu (0,3) relève le rappel à 0,69 là où le seuil naïf de 0,5 (celui de la première version du projet) donnait un F1 d'environ 0,50 ; le F1 atteint désormais 0,626, soit un gain d'environ douze points obtenu *sans toucher au modèle*, simplement par un protocole expérimental plus rigoureux. À l'inverse, l'arbre préfère un seuil élevé (0,6) qui filtre les feuilles incertaines et lui donne sa précision remarquable. Le seuil optimal est donc bien une propriété propre à chaque modèle, qu'il serait incorrect de fixer uniformément à 0,5.

8.3. Surapprentissage : un risque contrôlé

La comparaison des métriques d'entraînement et de test (`results/metrics.json`) montre des écarts très faibles : F1 de 0,623 (train) contre 0,626 (test) pour la régression logistique, 0,814 contre 0,817 pour l'arbre. Deux mécanismes y contribuent : les garde-fous de l'arbre (taille minimale des nœuds et des feuilles) et surtout la sélection de la profondeur sur la validation, qui a écarté la profondeur 9 (F1 de validation 0,817 contre 0,819 pour la profondeur 7) ; le début de surapprentissage a donc été détecté et évité par le protocole lui-même. Le k -NN présente l'écart le plus visible (AUC 0,907 en train contre 0,877 en test), comportement attendu d'une méthode à base d'instances évaluée sur ses propres exemples mémorisés ; l'écart reste modéré grâce au lissage d'un k élevé (31).

8.4. Le rappel, limite commune aux trois modèles

Le résultat le plus instructif n'est pas la hiérarchie des modèles mais leur plafond commun : environ 30 % des défauts ne sont détectés par aucun modèle, alors même que leurs familles algorithmiques sont très différentes. Cela suggère que la limite ne vient pas des algorithmes mais de l'*information disponible* : une partie des défauts n'est simplement pas prévisible à partir des onze variables du dossier de crédit (accidents de la vie, perte d'emploi postérieure à l'octroi, etc.). Pour une banque, ce constat est important : améliorer encore le modèle rapporterait moins que collecter des variables plus informatives.

8.5. Points forts et réserves sur la qualité des données

Au crédit du travail : un protocole sans fuite de données (paramètres de prétraitement appris sur le seul entraînement), une évaluation finale unique, des implémentations validées par une référence externe (écart d'AUC $\leq 5 \times 10^{-4}$), une expérience entièrement reproductible (graine fixe) et couverte par onze tests automatisés.

Côté réserves : l'imputation par la médiane de `loan_int_rate` (près de 10 % de valeurs manquantes) lisse l'information d'une variable probablement liée au risque ; le label encoding impose un ordre artificiel aux deux variables nominales, ce qui pénalise principalement la régression logistique, et une partie de son retard pourrait venir de là plutôt que de sa linéarité seule ; enfin, le déséquilibre des classes n'est traité que par le réglage du seuil, sans ré-échantillonnage ni pondération.

9. Limites et améliorations possibles

Les limites identifiées dans l'analyse critique dessinent directement les améliorations envisageables, classées ici de la plus simple à la plus structurante.

Encodage one-hot des variables nominales. Remplacer le label encoding de `loan_intent` et `person_home_ownership` par un encodage one-hot (une colonne binaire par modalité) supprimerait l'ordre artificiel entre modalités. Le bénéfice attendu concerne d'abord la régression logistique, qui pourrait attribuer un poids propre à chaque modalité ; le coût est minime (quelques colonnes supplémentaires).

Régularisation L2. Notre régression logistique ne comporte aucune pénalité sur les poids, là où scikit-learn en applique une par défaut. C'est l'une des sources du petit écart observé. Ajouter un terme $\lambda \|\mathbf{w}\|^2$ au coût (et $2\lambda \mathbf{w}$ au gradient) est immédiat et stabiliserait le modèle, notamment après un passage au one-hot qui augmente le nombre de colonnes.

Validation croisée k -fold. Le découpage unique 70/15/15, même stratifié et à graine fixe, fournit une estimation ponctuelle des performances. Une validation croisée à k blocs donnerait une moyenne et un écart-type pour chaque métrique, permettant de dire si l'écart entre deux modèles est significatif. Le coût, à savoir k entraînements au lieu d'un, resterait négligeable ici (le pipeline complet dure une vingtaine de secondes).

Recherche d'hyperparamètres élargie. Les grilles utilisées sont volontairement compactes (quatre valeurs par hyperparamètre). Une grille plus fine (profondeurs intermédiaires, `min_samples_leaf` variable, k pairs/impairs, seuils au pas de 0,05) pourrait apporter quelques points supplémentaires, au prix d'un temps de calcul encore raisonnable.

Forêt aléatoire. L'extension la plus naturelle du travail : entraîner plusieurs arbres CART sur des échantillons bootstrap avec sous-échantillonnage des variables, et moyenner leurs prédictions. Notre implémentation de CART constitue déjà la brique de base ; la forêt réduirait l'instabilité de l'arbre seul, sa principale faiblesse structurelle.

Traitement explicite du déséquilibre. Compléter le réglage du seuil par une pondération des classes dans la fonction de coût, ou par un ré-échantillonnage de l'entraînement (sur-échantillonnage des défauts ou sous-échantillonnage des remboursements), pourrait relever le rappel (la métrique critique) au prix d'un arbitrage à documenter sur la précision.

Choix du seuil par coûts métier. Le F1-score accorde le même poids à la précision et au rappel. Une banque préférerait minimiser un coût total explicite, de la forme $C =$

$c_{FN} \cdot FN + c_{FP} \cdot FP$ avec $c_{FN} \gg c_{FP}$: le seuil optimal s'en déduirait directement, et la comparaison des modèles se ferait en euros plutôt qu'en points de F1.

Visualisations complémentaires. Deux figures enrichiraient l'analyse : la courbe précision-rappel, plus informative que la ROC en présence de déséquilibre, et l'importance des variables de l'arbre (réduction totale de Gini par variable), qui relierait les performances aux intuitions métier de la section 3.

10. Conclusion

10.1. Bilan du travail réalisé

Ce projet a permis de construire un pipeline d'apprentissage automatique complet et reproductible pour la prédiction du risque de crédit bancaire : chargement et nettoyage de 32 581 dossiers, encodage des variables catégorielles, découpage stratifié en trois ensembles (70/15/15), imputation et normalisation sans fuite de données, entraînement de trois modèles de familles différentes implémentés from scratch en Python, sélection des hyperparamètres sur l'ensemble de validation, évaluation finale unique sur l'ensemble de test, et génération automatique des métriques (`results/metrics.json`) et des figures. L'ensemble s'exécute en une seule commande et une vingtaine de secondes, et la justesse des trois implémentations a été validée par comparaison avec scikit-learn (écarts d'AUC inférieurs ou égaux à 5×10^{-4}).

10.2. Réponse à la problématique

Oui, le risque de défaut est prédictible avec une fiabilité utile à partir des seules caractéristiques du dossier, et la comparaison désigne clairement **l'arbre de décision** comme le meilleur compromis : F1 de 0,817, AUC de 0,909 et une précision de 0,977 qui rend ses alertes presque toujours fondées, devant le k -NN (F1 0,683) et la régression logistique (F1 0,626). La réponse doit cependant être nuancée par le plafond commun des trois modèles : environ 30 % des défauts restent indétectés, limite qui semble tenir à l'information disponible dans les données plus qu'aux algorithmes eux-mêmes. Un déploiement réel exigerait donc soit des variables supplémentaires, soit l'acceptation explicite de ce risque résiduel.

10.3. Intérêt de la migration vers Python

La réécriture du projet, initialement développé en C, s'est révélée bien plus qu'une traduction. Le passage à Python et NumPy a divisé la taille du code par plusieurs fois et l'a rendu directement confrontable à une référence (scikit-learn), ce que la version C ne permettait qu'imparfaitement. Surtout, l'effort économisé sur la gestion de la mémoire a pu être réinvesti dans le protocole expérimental : ajout d'un ensemble de validation dédié (dont l'apport est mesurable : environ douze points de F1 gagnés sur la régression logistique par le seul réglage du seuil), ajout d'une troisième méthode, et couverture par des tests automatisés. La compréhension fine des algorithmes n'a rien perdu au change, puisque chaque méthode reste implémentée from scratch, équations à l'appui.

10.4. Bilan personnel d'apprentissage

Ce projet m'a d'abord appris ce que « implémenter from scratch » exige réellement : écrire une sigmoïde, c'est facile ; écrire une sigmoïde qui ne déborde pas numériquement, une recherche de seuil de Gini qui tient en quelques opérations vectorisées, ou un k -NN qui ne sature pas la mémoire, c'est là que se loge la vraie compréhension des algorithmes. J'ai ensuite mesuré à quel point le protocole expérimental compte autant que les modèles : la distinction validation/test, que je percevais comme une convention, a produit sous mes yeux un gain de douze points de F1 et détecté un début de surapprentissage. Enfin, travailler sur des classes déséquilibrées m'a définitivement guéri de lire une accuracy au premier degré : c'est la matrice de confusion, lue avec les coûts métier en tête, qui dit ce qu'un classifieur vaut vraiment.

Bibliographie

- [1] Leo Breiman, Jerome Friedman, Richard Olshen et Charles Stone. *Classification and Regression Trees*. Wadsworth & Brooks/Cole, 1984. Ouvrage fondateur de l'arbre de décision CART implémenté dans le projet.
- [2] Thomas M. Cover et Peter E. Hart. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1) :21-27, 1967. DOI : 10.1109/TIT.1967.1053964. Article fondateur de la méthode des k plus proches voisins.
- [3] Tom Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8) :861-874, 2006. DOI : 10.1016/j.patrec.2005.10.010. Référence sur la courbe ROC et l'AUC employées pour comparer les modèles.
- [4] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt et al. Array programming with NumPy. *Nature*, 585(7825) :357-362, 2020. DOI : 10.1038/s41586-020-2649-2. Bibliothèque de calcul numérique, base de toutes les implémentations *from scratch*.
- [5] David W. Hosmer et Stanley Lemeshow. *Applied Logistic Regression*. John Wiley & Sons, 2^e édition, 2000. DOI : 10.1002/0471722146. Ouvrage de référence sur la régression logistique.
- [6] John D. Hunter. Matplotlib : a 2D graphics environment. *Computing in Science & Engineering*, 9(3) :90-95, 2007. DOI : 10.1109/MCSE.2007.55. Bibliothèque utilisée pour générer toutes les figures du rapport.
- [7] laotse. Credit Risk Dataset - jeu de données de risque de crédit utilisé dans le projet (32 581 dossiers, licence CC0). Kaggle, 2020. <https://www.kaggle.com/datasets/laotse/credit-risk-dataset>. Consulté en juin 2026.
- [8] Wes McKinney. Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, pages 56-61, 2010. DOI : 10.25080/Majora-92bf1922-00a. Bibliothèque pandas, utilisée pour le chargement et la manipulation des données.
- [9] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort et al. Scikit-learn : machine learning in Python. *Journal of Machine Learning Research*, 12 :2825-2830, 2011. <https://jmlr.org/papers/v12/pedregosa11a.html>. Bibliothèque utilisée uniquement pour vérifier les implémentations *from scratch*.
- [10] Python Software Foundation. Python - langage de programmation utilisé pour l'ensemble de l'implémentation du projet. <https://www.python.org/>, 1991. Consulté en juin 2026.